

CoreOps Week-2: Assignments

PART 1: API Server Internals

Understanding API Request Flow

Step 1: Start API Proxy

```
kubectrl proxy --port=8080
```

Step 2: Explore API groups

```
curl localhost:8080/api
```

```
curl localhost:8080/apis
```

```
curl localhost:8080/metrics
```

Expected Learning

- Understand API groups → core / apps / batch
- Learn how kubectrl is just a client wrapper

Authorization Testing

```
kubectrl auth can-i create pods
```

```
kubectrl auth can-i delete nodes
```

Do → **Observe** → **Explain** model:

- Which verbs work
 - Why certain verbs require cluster-admin
 - RBAC chaining
-

Break the Scheduler

This teaches *control-plane partial failure*.

Step 1: Stop scheduler static pod

```
sudo mv /etc/kubernetes/manifests/kube-scheduler.yaml /root/
```

Step 2: Create a deployment

```
kubectl create deployment test --image=nginx
```

Expected Observation

```
kubectl get pods
```

```
test-xxxx → Pending forever
```

Explain

- Scheduler decides **where** a pod runs
- Without it, API server accepts requests, but workloads do not place



Restore Scheduler

```
sudo mv /root/kube-scheduler.yaml /etc/kubernetes/manifests/
```

Break Controller-Manager

```
sudo mv /etc/kubernetes/manifests/kube-controller-manager.yaml /root/
```

Scale deployment

```
kubectl scale deploy test --replicas=3
```

Expected

Replicas desired: 3

Replicas created: 1

Why

Controller Manager performs:

- Node lifecycle
 - ReplicaSet reconciliation
 - Endpoints update
- Break it → No updates.

Restore

```
sudo mv /root/kube-controller-manager.yaml /etc/kubernetes/manifests/
```



PART 2: etcd Internals

Take snapshot

```
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save snapshot.db
```

Break etcd

```
sudo systemctl stop kubelet
sudo mv /etc/kubernetes/manifests/etcd.yaml /root/
```

Expected:

- API server becomes read-only
- kubectl freezes or errors out

Restore etcd from snapshot

```
ETCDCTL_API=3 etcdctl snapshot restore snapshot.db \
  --data-dir=/var/lib/etcd-restore
```

Modify etcd.yaml → replace data-dir → restart kubelet.

PART 3: Kubelet & Node Internals

Observe node heartbeats

kubectl get nodes -w

Stop kubelet: `sudo systemctl stop kubelet`

Observe:

- Node flips → NotReady after 30 seconds

Kubelet logs

`sudo journalctl -u kubelet -f`

Kubelet pod reconciliation

Deploy: `kubectl run web --image=nginx`

Delete pod file: `sudo rm -rf /var/lib/kubelet/pods/<POD_UID>`

Observe: Kubelet re-creates runtime sandbox

PART 4: Networking & kube-proxy

Verify service routing

- `kubectl run nginx --image=nginx`
- `kubectl expose pod nginx --port=80`
- `kubectl run curl --image=busybox -it --rm -- sh`
- `wget -O- http://nginx`

Should work.

Break kube-proxy routing

`sudo iptables -F`

Try again: `wget -O- http://nginx`

Expected: fails.

Explain:

- kube-proxy programs netfilter tables
- flushing breaks service routing

Diagnose with iptables

`sudo iptables-save | grep KUBE`

PART 5: Upgrades, Static Pods, Version Skew

Crash API server intentionally

`sudo nano /etc/kubernetes/manifests/kube-apiserver.yaml`

Add fake flag: `--foo-bar-baz=123`

Expected:

- kube-apiserver crashloops
- kubectl stops working
- etcd is fine

How to recover: Remove flag → wait for kubelet to respawn pod

Version skew simulation

Downgrade kubeadm on worker: `sudo apt install kubelet=1.28.* kubeadm=1.28.* kubectl=1.28.*`

Observe warnings: `kubectl get nodes`

Static Pod Drift

Delete API server manifest: `sudo rm /etc/kubernetes/manifests/kube-apiserver.yaml`

kubelet removes pod → control plane immediately breaks.

Restore to fix.

FINAL WEEK-2 PROJECT

Everyone must build:

1. Perform control plane failures
2. Restore etcd
3. Debug kubelet
4. Break + fix kube-proxy routing
5. Create taints, affinity, PDB
6. Produce a 2-page operations RCA

